

REPORTE TÉCNICO

Creación y sincronización de un repositorio GitHub

Detección de bordes de una imagen mediante Python

Materia:

Sistemas Operativos

Docente:

Hazem Álvarez Rodriguez

Alumna:

Galilea Vazquez Lira

Desarrollo de un proyecto utilizando CMD, Git, GitHub, Python y OpenCV

Agosto de 2026

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Herramientas utilizadas	3
4. Desarrollo de la práctica	3
4.1. Creación del repositorio en GitHub	3
4.2. Creación de la carpeta mediante CMD	3
4.3. Inicialización del repositorio local	4
4.4. Vinculación con GitHub	4
4.5. Verificación de Python	4
4.6. Instalación de OpenCV	5
4.7. Selección de la imagen	5
4.8. Desarrollo del programa	5
4.9. Ejecución del programa	6
5. Resultados	6
5.1. Evidencia del código	7
5.2. Resultado de la ejecución	7
5.3. Archivos generados	9
6. Control de versiones y publicación en GitHub	9
7. Estructura final del proyecto	9
8. Conclusiones	10
9. Referencias	10

1. Introducción

En la presente práctica se realizó un proyecto utilizando herramientas relacionadas con la administración de sistemas, control de versiones y programación. El propósito fue crear un repositorio en GitHub, generar una carpeta de trabajo mediante la línea de comandos de Windows (CMD), vincular dicha carpeta con el repositorio remoto y desarrollar un programa en Python capaz de detectar los bordes de una imagen.

Para el desarrollo del programa se utilizó la biblioteca OpenCV, la cual proporciona diferentes herramientas para el procesamiento digital de imágenes. En este caso se utilizó el algoritmo de Canny para identificar los bordes presentes en una imagen de un edificio.

Además del desarrollo del programa, se practicó el uso de Git para administrar los archivos del proyecto. Se utilizaron comandos como `git init`, `git remote`, `git add`, `git commit` y `git push`, permitiendo almacenar finalmente el proyecto en GitHub.

La práctica permitió integrar conocimientos de sistemas operativos, línea de comandos, programación y control de versiones en un mismo proyecto.

2. Objetivos

2.1. Objetivo general

Crear y publicar un proyecto utilizando Git y GitHub, desarrollando un programa en Python que permita obtener los bordes de una imagen mediante técnicas de procesamiento digital de imágenes.

2.2. Objetivos específicos

- Crear un repositorio nuevo en GitHub.
- Crear una carpeta de trabajo utilizando CMD.
- Inicializar un repositorio local mediante Git.
- Vincular el repositorio local con el repositorio remoto de GitHub.
- Verificar la instalación de Python.
- Instalar y utilizar la biblioteca OpenCV.
- Desarrollar un programa en Python para detectar los bordes de una imagen.
- Guardar el resultado generado por el programa.
- Crear un commit con los archivos del proyecto.
- Subir los archivos y resultados al repositorio de GitHub.

3. Herramientas utilizadas

Para realizar la práctica se utilizaron las siguientes herramientas:

- **Windows CMD:** utilizado para ejecutar comandos y administrar la carpeta del proyecto.
- **Git:** utilizado como sistema de control de versiones para administrar los archivos y registrar los cambios realizados.
- **GitHub:** utilizado como repositorio remoto para almacenar y publicar el proyecto.
- **Python 3.11.9:** lenguaje de programación utilizado para desarrollar el programa de detección de bordes.
- **OpenCV:** biblioteca utilizada para el procesamiento de imágenes y detección de bordes.
- **Bloc de notas:** utilizado para crear y editar el archivo de código Python.

4. Desarrollo de la práctica

4.1. Creación del repositorio en GitHub

El primer paso consistió en crear un nuevo repositorio en GitHub. El repositorio utilizado durante la práctica recibió el nombre:

DevOps_VazquezGalilea

Este repositorio se utilizó posteriormente como repositorio remoto para almacenar los archivos desarrollados durante la práctica.

El repositorio permitió centralizar el código fuente, la imagen original y el resultado obtenido después de ejecutar el programa.

4.2. Creación de la carpeta mediante CMD

Después de crear el repositorio, se abrió el Símbolo del sistema de Windows (CMD).

Inicialmente se accedió a la carpeta Documentos utilizando el comando:

```
1 cd /d "%USERPROFILE%\Documents"
```

Posteriormente se creó la carpeta correspondiente al proyecto mediante:

```
1 mkdir proyecto_bordes
```

Finalmente se ingresó a la carpeta creada:

```
1 cd proyecto_bordes
```

La carpeta utilizada durante la práctica quedó ubicada en:

C:\Users\ekt\OneDrive\Documentos\proyecto_bordes

4.3. Inicialización del repositorio local

Una vez dentro de la carpeta del proyecto, se utilizó el siguiente comando:

```
1 git init
```

Este comando permitió inicializar un repositorio local de Git dentro de la carpeta `proyecto_bordes`.

Después de ejecutar el comando, Git creó internamente la carpeta de configuración necesaria para comenzar a realizar el control de versiones del proyecto.

4.4. Vinculación con GitHub

Posteriormente se estableció la conexión entre el repositorio local y el repositorio remoto de GitHub mediante:

```
1 git remote add origin https://github.com/galileaLira/  
  DevOps_VazquezGalilea.git
```

Para verificar que la conexión se hubiera realizado correctamente se utilizó:

```
1 git remote -v
```

El resultado permitió comprobar que el repositorio local tenía configurado correctamente el repositorio remoto denominado `origin`.

4.5. Verificación de Python

Antes de desarrollar el programa se comprobó que Python estuviera instalado correctamente en el equipo mediante:

```
1 python --version
```

La versión utilizada durante la práctica fue:

Python 3.11.9

4.6. Instalación de OpenCV

Para realizar el procesamiento de imágenes se utilizó la biblioteca OpenCV.

La instalación se realizó desde CMD mediante:

```
1 pip install opencv-python
```

Después de ejecutar el comando se comprobó que OpenCV estuviera disponible para Python.

4.7. Selección de la imagen

Para realizar la prueba se utilizó una imagen de un edificio, almacenada dentro de la carpeta del proyecto con el nombre:

edificio.jpg

La imagen fue utilizada como entrada para el programa de procesamiento.

4.8. Desarrollo del programa

El programa fue desarrollado en Python utilizando la biblioteca OpenCV.

El archivo creado recibió el nombre:

bordes.py

El código utilizado fue el siguiente:

```
1 import cv2
2
3 # Cargar la imagen original
4 imagen = cv2.imread("edificio.jpg")
5
6 # Convertir la imagen a escala de grises
7 gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
8
9 # Detectar los bordes de la imagen
10 bordes = cv2.Canny(gris, 100, 200)
11
12 # Guardar la imagen resultante
13 cv2.imwrite("bordes_edificio.jpg", bordes)
14
15 # Mostrar la imagen con los bordes
16 cv2.imshow("Bordes del edificio", bordes)
17
```

```
18 # Esperar hasta que se presione una tecla
19 cv2.waitKey(0)
20
21 # Cerrar las ventanas de OpenCV
22 cv2.destroyAllWindows()
```

Listing 1: Código desarrollado para detectar los bordes de la imagen.

El programa comienza importando la biblioteca OpenCV mediante `import cv2`. Posteriormente se carga la imagen original utilizando la función `cv2.imread()`.

La imagen se convierte a escala de grises mediante `cv2.cvtColor()`, debido a que el algoritmo de detección de bordes puede trabajar con la intensidad de los píxeles.

Después se utiliza la función `cv2.Canny()` para detectar los bordes. En este caso se utilizaron los valores 100 y 200 como umbrales.

Finalmente, el resultado se guarda mediante `cv2.imwrite()` con el nombre `bordes_edificio.jpg` y se muestra en pantalla mediante `cv2.imshow()`.

4.9. Ejecución del programa

Una vez guardado el código, el programa fue ejecutado desde CMD utilizando:

```
1 python bordes.py
```

Al ejecutarse correctamente, se mostró una ventana con la imagen procesada y los bordes detectados.

La ejecución confirmó que el programa podía leer la imagen, procesarla y generar el resultado esperado.

5. Resultados

Como resultado de la ejecución se generó una nueva imagen denominada:

`bordes_edificio.jpg`

Esta imagen contiene los bordes detectados de la imagen original mediante el algoritmo de Canny.

La estructura final de la carpeta del proyecto quedó de la siguiente manera:

```
proyecto_bordes/
|
|-- bordes.py
|-- edificio.jpg
'-- bordes_edificio.jpg
```

El archivo `edificio.jpg` corresponde a la imagen original, `bordes.py` contiene el programa desarrollado y `bordes_edificio.jpg` corresponde al resultado generado.

5.1. Evidencia del código

A continuación se presenta la evidencia del código utilizado para realizar el procesamiento de la imagen.

```
1  Importación CV2
2
3  # Cargar la imagen original
4  Imagen = CV2.imread("edificio.jpg")
5
6  # Convertir la imagen a escala de grises
7  Gris = CV2.cvtColor(Imagen, CV2.COLOR_BGR2GRAY)
8
9  # Detectar los bordes de la imagen
10 Bordes = CV2.Astuto(Gris, 100, 200)
11
12 # Guardar la imagen resultante
13 CV2.imwrite("bordes_edificio.jpg", Bordes)
14
15 # Mostrar la imagen con los bordes
16 CV2.imshow("Bordes del edificio", Bordes)
17
18 # Esperar hasta que se presione una tecla
19 CV2.waitKey(0)
20
21 # Cerrar las ventanas de OpenCV
22 CV2.destroyAllWindows()
```

Figura 1: Código en Python utilizado para la detección de bordes.

Como se observa en la Figura 1, el programa utiliza OpenCV para cargar la imagen, convertirla a escala de grises, detectar sus bordes y guardar el resultado.

5.2. Resultado de la ejecución

La ejecución del programa produjo una imagen en la que se pueden observar los principales contornos y estructuras presentes en el edificio.



Figura 2: Resultado de la detección de bordes del edificio.

La Figura 2 muestra el resultado obtenido después de aplicar el algoritmo de Canny sobre la imagen original.

5.3. Archivos generados

Durante la práctica se generaron tres archivos principales:

- `bordes.py`: programa utilizado para procesar la imagen.
- `edificio.jpg`: imagen original utilizada como entrada.
- `bordes_edificio.jpg`: imagen resultante de la detección de bordes.

6. Control de versiones y publicación en GitHub

Después de comprobar que el programa funcionaba correctamente, se utilizó Git para agregar los archivos al repositorio local.

Primero se verificó el estado del repositorio mediante:

```
1 git status
```

Posteriormente se agregaron los archivos utilizando:

```
1 git add .
```

Después se creó un commit para registrar los cambios:

```
1 git commit -m "Agrega programa de deteccion de bordes"
```

El commit permitió registrar los tres archivos creados durante la práctica.

Finalmente, se enviaron los archivos al repositorio remoto de GitHub mediante:

```
1 git push -u origin master
```

La operación se realizó correctamente y GitHub confirmó la creación de la rama remota `master`.

De esta manera, el proyecto quedó almacenado en el repositorio remoto de GitHub.

7. Estructura final del proyecto

Después de completar todas las actividades, la estructura del proyecto quedó de la siguiente manera:

```
DevOps_VazquezGalilea/  
|  
|-- bordes.py  
|-- edificio.jpg  
'-- bordes_edificio.jpg
```

El repositorio contiene tanto el código fuente como los archivos utilizados y el resultado generado por el programa.

8. Conclusiones

La realización de esta práctica permitió integrar diferentes herramientas relacionadas con los sistemas operativos, la programación y el control de versiones.

En primer lugar, se aprendió a utilizar CMD para desplazarse entre directorios y crear una carpeta de trabajo desde la línea de comandos. Posteriormente se utilizó Git para inicializar un repositorio local y vincularlo con un repositorio remoto de GitHub.

También se desarrolló un programa en Python utilizando la biblioteca OpenCV. El programa permitió cargar una imagen, convertirla a escala de grises y aplicar el algoritmo de Canny para detectar los bordes presentes en la imagen.

El resultado obtenido permitió comprobar que el programa funcionaba correctamente, ya que se generó una nueva imagen con los bordes detectados del edificio.

Finalmente, mediante los comandos `git add`, `git commit` y `git push`, se logró almacenar el código, la imagen original y el resultado dentro del repositorio de GitHub.

En conclusión, la práctica permitió comprender de manera práctica el funcionamiento básico de Git y GitHub, así como la utilidad de la línea de comandos y de Python para realizar tareas de procesamiento de imágenes.

9. Referencias

- Git. Documentación oficial. <https://git-scm.com/doc>
- GitHub Docs. Documentación oficial. <https://docs.github.com/>
- Python Software Foundation. Documentación oficial de Python. <https://docs.python.org/3/>
- OpenCV. Documentación oficial. <https://docs.opencv.org/>